SoftwareX 10 (2019) 100265

Contents lists available at ScienceDirect

SoftwareX

journal homepage: www.elsevier.com/locate/softx

Original software publication

RTransferEntropy — Quantifying information flow between different time series using effective transfer entropy

Simon Behrendt [a], Thomas Dimpfl [b,*], Franziska J. Peter [a], David J. Zimmermann [c]

[a] Zeppelin University, Department of Empirical Finance and Econometrics, 88045 Friedrichshafen, Germany
[b] University of Tübingen, Faculty of Economics and Social Sciences, School of Business and Economics, Department of Statistics, Econometrics, and Empirical Economic Research, 72074 Tübingen, Germany
[c] University Witten/Herdecke, Department of Banking and Finance, 58448 Witten, Germany

ARTICLE INFO

Article history:
Received 14 March 2019
Received in revised form 21 June 2019
Accepted 21 June 2019

Keywords:
Shannon transfer entropy
Rényi transfer entropy
Effective transfer entropy
Bootstrap inference
R

ABSTRACT

This paper shows how to quantify and test for the information flow between two time series with Shannon transfer entropy and Rényi transfer entropy using the *R* package *RTransferEntropy*. We discuss the methodology, the bias correction applied to calculate effective transfer entropy and outline how to conduct statistical inference. Furthermore, we describe the package in detail and demonstrate its functionality by means of several simulated processes and present an application to financial time series.

Code metadata

Current code version	v 0.2.8
Permanent link to code/repository used for this code version	https://github.com/ElsevierSoftwareX/SOFTX_2019_13
Code Ocean compute capsule	–
Legal Code License	GPL-3
Code versioning system used	git
Software code languages, tools, and services used	R, C++, GitHub, TravisCI
Compilation requirements, operating environments & dependencies	64-bit operating system, R (version $\geq$ 3.1.2), R packages: future, Rcpp
If available Link to developer documentation/manual	https://github.com/BZPaper/RTransferEntropy
Support email for questions	thomas.dimpfl@uni-tuebingen.de

Software metadata

Current software version	v 0.2.8
Permanent link to executables of this version	https://github.com/BZPaper/RTransferEntropy
Legal Software License	GPL-3
Computing platforms/Operating Systems	Linux, MacOS, Microsoft Windows
Installation requirements & dependencies	64-bit operating system, R (version $\geq$ 3.1.2), R packages: future, Rcpp
If available, link to user manual – if formally published include a reference to the publication in the reference list	https://github.com/BZPaper/RTransferEntropy
Support email for questions	thomas.dimpfl@uni-tuebingen.de

1. Introduction

Quantifying the information transfer between different time series is of great interest in various research areas. This endeavor

* Corresponding author.
E-mail address: thomas.dimpfl@uni-tuebingen.de (T. Dimpfl).

https://doi.org/10.1016/j.softx.2019.100265

commonly relies on measures derived from subject-specific assumptions and restrictions concerning the underlying stochastic processes, or dynamic, theoretical models. Transfer entropy [1], which is a nonparametric measure of asymmetric information transfer between time series, provides a good alternative. It requires only few assumptions concerning the underlying stochastic processes which allows for a widespread use. It has been applied to detect differences between Electroencephalography (EEG) data from healthy and epileptic individuals [2], to analyze the Antarctic Circumpolar Wave [3], or to examine information transmission between financial markets [4]. It is also used in biomedical engineering [5,6], neuroscience [7–9] or to extract information from social media data [10]. See [11] for more information on statistical and theoretical background, estimation methods, and applications of transfer entropy.

Our package *RTransferEntropy* [12] offers a user-friendly tool to calculate the Shannon and Rényi transfer entropy in *R*. A core aspect of the package is to allow statistical inference and hypothesis testing in the context of transfer entropy. Furthermore, it offers various possibilities for symbolic encoding of the data, based on either a priori defined bins or quantiles of the empirical distribution of the data.

This paper is organized as follows: Section 2 introduces Shannon and Rényi transfer entropy, the bias correction, and the bootstrap used to conduct statistical inference. Section 3 describes the package and its functionality and Section 4 compares it to existing software. Sections 5 and 6 illustrate the use of the package by means of simulated time series and empirical data, respectively. Section 7 concludes.

2. Estimating directional dependencies using transfer entropy

2.1. Transfer entropy and Rényi transfer entropy

Transfer entropy is rooted in information theory and based on the concept of Shannon entropy as a measure of uncertainty. In 1928, Hartley [13] introduced a measure for information calculated as the logarithm of the number of all possible symbol sequences that can occur based on a specific probability distribution (see the example in [14]). Consider a probability distribution where the different outcomes occur with different probabilities p_j. The average information per symbol is defined as $H = \sum_{j=1}^{n} p_j \log_2(\frac{1}{p_j})$ bits, where n is the number of distinct symbols associated with the probabilities p_j.

In 1948, Shannon [15] introduced the concept of entropy (later referred to as Shannon entropy), which states that for a discrete random variable J with probability distribution $p(j)$, the average number of bits required to optimally encode independent draws can be calculated as

$$H_J = -\sum_j p(j) \log_2 p(j). \tag{1}$$

In the following, *log* denotes the base 2 logarithm.[1] Shannon's formula measures uncertainty, which increases with the number of bits needed to optimally encode a sequence of realizations of J. In order to measure the information flow between two time series, Shannon entropy is coupled with the concept of Kullback–Leibler distance [16] under the assumption that the underlying processes are Markov processes [1]. Let I and J denote two discrete random variables with marginal probability distributions $p(i)$ and $p(j)$, joint probability $p(i, j)$, whose dynamical structures correspond to a stationary Markov process of order k (process I) and l (process J). The Markov property implies that the probability to observe I at time $t+1$ in state i conditional on the k previous observations is $p(i_{t+1}|i_t, \ldots, i_{t-k+1}) = p(i_{t+1}|i_t, \ldots, i_{t-k})$. The average number of bits needed to encode the observation in $t+1$, once the previous k values are known, is given by

$$h_I(k) = -\sum_i p(i_{t+1}, i_t^{(k)}) \log p(i_{t+1}|i_t^{(k)}), \tag{2}$$

where $i_t^{(k)} = (i_t, \ldots, i_{t-k+1})$ (analogously for process J). In the bivariate case information flow from process J to process I is measured by quantifying the deviation from the generalized Markov property $p(i_{t+1}|i_t^{(k)}) = p(i_{t+1}|i_t^{(k)}, j_t^{(l)})$, relying on the Kullback–Leibler distance. The formula for Shannon transfer entropy [1] is given by

$$T_{J\to I}(k, l) = \sum p(i_{t+1}, i_t^{(k)}, j_t^{(l)}) \log \frac{p(i_{t+1}|i_t^{(k)}, j_t^{(l)})}{p(i_{t+1}|i_t^{(k)})}, \tag{3}$$

where $T_{J\to I}$ measures the information flow from J to I. $T_{I\to J}$, as a measure for the information flow from I to J, can be derived analogously. The dominant direction of the information flow can be inferred by calculating the difference between $T_{J\to I}$ and $T_{I\to J}$.

Transfer entropy can also be based on Rényi entropy [17,18]. Rényi entropy depends on a weighting parameter q and can be calculated as

$$H_J^q = \frac{1}{1-q} \log \sum_j p^q(j)$$

with $q > 0$. For $q \to 1$, Rényi entropy converges to Shannon entropy.[2] For $0 < q < 1$ events that have a low probability receive more weight, while for $q > 1$ the weights favor outcomes j with a higher initial probability. Consequently, Rényi entropy allows to emphasize different areas of a distribution, depending on the parameter q.

Using the escort distribution [19] $\phi_q(j) = \frac{p^q(j)}{\sum_j p^q(j)}$ with $q > 0$ to normalize the weighted distributions, Rényi transfer entropy [18] is derived as

$$RT_{J\to I}(k, l) = \frac{1}{1-q} \log \frac{\sum_i \phi_q(i_t^{(k)}) p^q(i_{t+1}|i_t^{(k)})}{\sum_{i,j} \phi_q(i_t^{(k)}, j_t^{(l)}) p^q(i_{t+1}|i_t^{(k)}, j_t^{(l)})}. \tag{4}$$

Note that the calculation of Rényi transfer entropy can result in negative values. In such a situation, knowing the history of J reveals even greater uncertainty than would otherwise be indicated by only knowing the history of I alone. For more details see [18].

2.2. Diagnostics and inference

The transfer entropy estimates are biased in small samples [20]. A bias correction is possible and used to calculate effective transfer entropy [20] as

$$ET_{J\to I}(k, l) = T_{J\to I}(k, l) - T_{J_{shuffled}\to I}(k, l), \tag{5}$$

where $T_{J_{shuffled}\to I}(k, l)$ indicates the transfer entropy using a shuffled version of the time series J. Shuffling implies randomly drawing values from the observed time series J and realigning them to generate a new time series, which destroys the time series dependencies of J as well as the statistical dependencies between J and I. As a result, $T_{J_{shuffled}\to I}(k, l)$ converges to zero with increasing sample size and any nonzero value of $T_{J_{shuffled}\to I}(k, l)$ is due to small sample effects. To derive a consistent estimator, shuffling is repeated and the average of the resulting shuffled transfer entropy estimates across all replications serves as an estimator for the small sample bias, which is subsequently subtracted

[1] The choice of the logarithm's base only impacts on the unit of measurement. Base 2 logarithm indicates bits, base 10 digits, and the base of the natural logarithm yields nats.

[2] A proof is provided in the Appendix.

from the Shannon or Rényi transfer entropy estimate to obtain a bias corrected effective transfer entropy estimate.

To assess the statistical significance of the transfer entropy estimates, as given by Eq. (3), we rely on a Markov block bootstrap [14]. In contrast to shuffling, it preserves the dependencies within the variables J and I, but eliminates the statistical dependencies between them. Repeated estimation of transfer entropy then provides the distribution of the estimates under the null hypothesis of no information flow. The associated p value is given by $1-\hat{q}_T$, where $\hat{q}_T$ denotes the quantile of the simulated distribution that is determined by the respective transfer entropy estimate.

The calculation of transfer entropy is based on discrete data. Therefore, continuous data have to be discretized. This can be achieved by symbolic encoding, i.e. partitioning the data into a finite number of bins. Denote the bounds specified for the n bins by $q_1, q_2, \ldots, q_{n-1}$, where $q_1 < q_2 < \cdots < q_{n-1}$, and consider an observed time series denoted by y_t. The encoded time series is obtained as

$$S_t = \begin{cases} 1 & \text{for } y_t \leq q_1 \\ 2 & \text{for } q_1 < y_t < q_2 \\ \vdots & \vdots \\ n-1 & \text{for } q_{n-2} < y_t < q_{n-1} \\ n & \text{for } y_t \geq q_{n-1}. \end{cases} \tag{6}$$

The choice of the bins should be motivated by the distribution of the data. In the case of financial return data, tail observations are commonly of particular interest and therefore the binning is usually based on empirical quantiles such that the left and right tail observations are selected into separate bins. In the case of three bins, this can be achieved, for example, by using the 5% and 95% empirical quantiles of the return data as lower and upper bounds for binning, resulting in a symbolic encoding where the first (third) bin includes the extreme negative (positive) returns. Based on symbolic encoding and by making use of the chain rule in order to write conditional probabilities as fractions of joint probabilities, the probabilities in Eqs. (3) and (4) can then easily be computed by the relative frequencies of all the possible outcomes. While determining the bins in the above mentioned way may be a reasonable approach for many empirical applications, other ways to determine $q_1, q_2, \ldots, q_{n-1}$ are possible. For example, one may specify a number of equal width bins or set predetermined limits for each bin.

3. The *RTransferEntropy* package

The *R* package *RTransferEntropy* offers straightforward routines to quantify and test the information flow between two time series measurements with Shannon and Rényi transfer entropy. The main function is `transfer_entropy()`, which returns the values for entropy estimates, the effective transfer entropy estimates, standard errors and p-values, an indication of statistical significance, and quantiles of the bootstrap samples. We also provide the functions `calc_te()` and `calc_ete()` to calculate the transfer entropy or the effective transfer entropy for a single direction, without bootstrapping standard errors and p value. Time-consuming functions are written in *C++* using the *Rcpp* package [21]. Furthermore, we provide parallel computing functionality using the *future* package [22].

In the following, We describe the usage of `transfer_entropy()` and its options.

```
R> transfer_entropy(x, y, lx = 1, ly = 1, q = 0.1,
R>     entropy = c("Shannon", "Renyi"), shuffles = 100,
R>      type = c("quantiles", "bins", "limits"),
R>    quantiles = c(5, 95), bins = NULL, limits = NULL,
R>   nboot = 300, burn = 50, quiet = FALSE, seed = NULL)
```

The main arguments are `x` and `y`, two time series with equidistant spacing of observations. Both `x` and `y` are required to be numeric vectors of the same length. The respective number of lags is specified by `lx` and `ly` with a default value of 1. Note that choosing `lx` and `ly` is a non-trivial task and generally dependent on the time series that are being analyzed, which is why *RTransferEntropy* does not impose a certain algorithm on the user. For time series with strong deterministic structure the approaches discussed in [23,24] might be reasonable choices, while for time series generated by stochastic processes some approaches are outlined in [14,25]. The user should first determine appropriate values for `lx` and `ly` and then proceed to estimate the transfer entropy with the `transfer_entropy()` function.[3] To facilitate the interpretation of results, `lx` and `ly` should be of the same order (see [1]). The type of transfer entropy to be estimated can be chosen with `entropy`. The default is Shannon transfer entropy. To calculate effective transfer entropy, the number of `shuffles` to be used has to be specified (default 100). To obtain reliable estimates, one should choose relatively large numbers. Furthermore, the discretization has to be specified. Via `type` one can choose whether to base discretization on quantiles of the empirical distribution of the observed time series (the default), on a number of bins with equal width, or on user specified limits for the bins. The quantiles (number of bins or limits) have to be specified. By default, the 5% and 95% quantiles of the empirical distribution are used.

Statistical inference is based on bootstrap samples of the transfer entropy estimates, not the effective transfer entropy estimates. The number of bootstrap replications is set with `nboot` (default 300). The number of observations that are dropped from the beginning of the bootstrapped Markov chain is governed by `burn` (default 50). If `quiet` is set to `FALSE` (default), the function gives feedback on the progress of the calculations. Lastly, a seed value for the pseudo-random number generator can be provided with the `seed` argument.[4]

Since bootstrapping comes along with an increased computational burden, parallel computing is supported using the *future* package. By default, code is executed in a sequential manner. To enable parallel execution, the user must activate a `future::plan()` by calling `plan(multiprocess)` (see [22] for further details). The following code allows us to use all available cores on the current machine. The used number of cores can be limited by specifying the number n of workers as `plan(multiprocess(workers = n))`.

```
R> library(future)
R> plan(multiprocess)
```

To demonstrate the output of `transfer_entropy()` we provide a simple example. Let us consider a linear relationship between two random variables X and Y, where Y depends on X with one lag and X is independent of Y, such that

$$\begin{aligned} x_t &= 0.2x_{t-1} + \varepsilon_{x,t}, \\ y_t &= x_{t-1} + \varepsilon_{y,t}, \end{aligned} \tag{7}$$

with $\varepsilon_{x,t}$ and $\varepsilon_{y,t}$ being normally distributed with a mean of 0 and a standard deviation of 2. In this case, X serves as a predictor for Y, but not vice versa. We simulate 10,000 observations of these processes as follows:

[3] For example, algorithms for time series exhibiting deterministic structure are implemented in the packages *tseriesChaos* [26] and *nonlinearTseries* [27].

[4] Internally, `transfer_entropy()` will simply call `set.seed(seed)`. This functionality is preserved to make reproducibility under parallel execution easier for the user.

```
R> set.seed(12345)
R> n <- 10000
R> x <- rep(0, n + 1)
R> y <- rep(0, n + 1)
R>
R> for (i in seq(n)) {
R+   x[i + 1] <- 0.2 * x[i] + rnorm(1, 0, 2)
R+   y[i + 1] <- x[i] + rnorm(1, 0, 2)
R+ }
R>
R> x <- x[-1]
R> y <- y[-1]
```

Subsequently, we estimate Shannon transfer entropy with the default settings for all function arguments (except the seed).

```
R> shannon_te <- transfer_entropy(x = x, y = y, seed =
12345)

Shannon's entropy on 8 cores with 100 shuffles.
 x and y have length 10000 (0 NAs removed)
 [calculate] X->Y transfer entropy
 [calculate] Y->X transfer entropy
 [bootstrap] 300 times
Done - Total time 14.69 seconds
```

The output of `transfer_entropy()` below summarizes all important information and is highlighted for illustrative purposes. In the upper (red) part of the output table the (effective) transfer entropy is given in the `TE` (`Eff. TE`) column for each direction of the information flow. Standard errors, *p*-values and an indication for statistical significance (see the explanation at the bottom) are given in columns four to six. These quantities are based on the bootstrap samples, whose quantiles are provided in the lower (blue) part of the output table for both directions of the information flow. The `TE` estimates are compared to the bootstrap samples in order to calculate *p*-values, as discussed in Section 2. From the output below, we can see that there is a significant information flow from X to Y but not vice versa (as expected, see Eq. (7)).

```
R> shannon_te

Shannon Transfer Entropy Results:
------------------------------------------------
Direction     TE  Eff. TE Std. Err.  p-value  sig
------------------------------------------------
   X->Y   0.0902  0.0893  0.0003   0.0000   ***
   Y->X   0.0006  0.0000  0.0003   0.8033
------------------------------------------------
Bootstrapped TE Quantiles (300 replications):
------------------------------------------------
Direction    0%    25%    50%    75%   100%
------------------------------------------------
  X->Y 0.0003 0.0006 0.0009 0.0011 0.0023
  Y->X 0.0003 0.0007 0.0009 0.0011 0.0029
------------------------------------------------
Number of Observations: 10000
------------------------------------------------
p-values: < 0.001 '***', < 0.01 '**', < 0.05
'*', < 0.1 '.'
```

4. Comparison to existing transfer entropy tools

This section gives a brief overview of existing tools that allow the calculation of transfer entropy metrics. The tools we consider in this comparison are listed in Table 1. With the exception of the MuTE tool [28], all tools are licensed under the GPL v3 open source license and are free to use. The JIDT [29] and IDTxl [30] are still under active maintenance, while Trentool [31] and MuTE [28] are out of date and seem to be no longer maintained. Both JIDT and IDTxl are toolboxes in the general area of information dynamics and not specific to transfer entropy, whereas our package focuses on transfer entropy measures.

While all tools allow the calculation of Shannon transfer entropy, *RTransferEntropy* is the only tool that allows the calculation of Rényi and effective transfer entropy. Furthermore, *RTransferEntropy* aims to be an easy-to-use and light on dependency tool that allows the calculation of transfer entropy metrics in the *R* ecosystem.

5. Applications to simulated processes

Consider again the above example where the results show a significant information flow from X to Y but not vice versa. Similar conclusions could be drawn from using a vector autoregressive (VAR) model and testing for Granger causality [32]. However, the main advantage of using transfer entropy is that it is not limited to linear relationships. Consider the following nonlinear relation between X and Y, where only Y depends on X:

$$\begin{aligned} x_t &= 0.2x_{t-1} + \varepsilon_{x,t}, \\ y_t &= \sqrt{|x_{t-1}|} + \varepsilon_{y,t}, \end{aligned} \tag{8}$$

with $\varepsilon_{x,t}$ and $\varepsilon_{y,t}$ being standard normally distributed. We simulate these processes, discarding the first 200 observations as a burn-in period in *R* as follows:

```
R> set.seed(12345)
R> n <- 10000
R> x <- rep(0, n + 200)
R> y <- rep(0, n + 200)
R>
R> x[1] <- rnorm(1, 0, 1)
R> y[1] <- rnorm(1, 0, 1)
R>
R> for (i in 2:(n + 200)) {
R+   x[i] <- 0.2 * x[i - 1] + rnorm(1, 0, 1)
R+   y[i] <- sqrt(abs(x[i - 1])) + rnorm(1, 0, 1)
R+ }
R>
R> x <- x[-(1:200)]
R> y <- y[-(1:200)]
```

The focus of this example is on Shannon transfer entropy. We use the standard settings of the `transfer_entropy()` function, using one lag to calculate the transfer entropy.

```
R> shannon_te2 <- transfer_entropy(x = x, y = y,
seed = 12345)
```

The resulting Shannon transfer entropy estimate, as shown in Table 2, indicates that there is a significant information flow from X to Y, but not in the other direction. In the same situation, a VAR model would not reveal any relationship between X and Y. Using the package *vars* [33] and one lag (the true lag structure) delivers the following estimates of the VAR(1) for the dependent variable Y.

Table 1
Comparison of toolboxes.

Toolbox	Source	Language	Available functions	Testing	License	Version
Trentool	[31]	Matlab	Transfer Entropy	Yes	GPL v3	3.4.3
JIDT	[29]	Java*	Transfer Entropy, Mutual Information; Active Information Storage	No	GPL v3	1.5
MuTE	[28]	Matlab	Transfer Entropy; Mutual Information	No	–	–
IDTxl	[30]	Python	(Multivariate) Transfer Entropy; (Multivariate) Mutual Information; Active Information Storage; Partial Information Decomposition	Yes	GPL v3	1
RTRansferEntropy		R	(Effective) Transfer Entropy; (Effective) Rényi Transfer Entropy	Yes	GPL v3	0.2.8

Note. The table provides an overview of the main features of existing toolboxes that allow the calculation of transfer entropy.
* Provides bindings to Matlab, Python, R, Julia, and Clojure.

Table 2
Shannon transfer entropy result.

Panel A: Transfer entropy estimates					
Direction	TE	Eff. TE	Std. Err.	*p*-value	sig
X → Y	0.0125	0.0115	0.0003	0.0000	***
Y → X	0.0005	0.0000	0.0003	0.8900	
Panel B: Bootstrapped TE quantiles					
Direction	0%	25%	50%	75%	100%
X → Y	0.0002	0.0006	0.0008	0.0010	0.0023
Y → X	0.0002	0.0007	0.0009	0.0011	0.0024

Note. The table presents the transfer entropy estimates based on data simulated according to Eq. (8). The estimation (Panel A) is based on 1000 observations. Bootstrapped transfer entropy quantiles (Panel B) are based on 300 bootstrap replications.
***(**, *) indicates significance on a 0.1% (1%, 5%) significance level.

Table 3
Nonlinear dependence VAR results.

	X	*Y*
const	-0.014	0.824***
	(0.013)	(0.013)
X.L1	0.192***	0.002
	(0.010)	(0.011)
Y.L1	0.000	0.008
	(0.013)	(0.010)

Note. The table presents the estimation results from the VAR estimation based on data simulated according to Eq. (8). Standard errors are provided in parentheses.
*** (**, *) indicates significance on a 1% (5%, 10%) significance level.

```
R> library(vars)
R> varfit <- VAR(cbind(x, y), p = 1, type = "const")
```

The results of the estimation are presented in Table 3. The VAR cannot detect the nonlinear dependence of Y on X, which leads to a parameter estimate `X.L1` that is not statistically significant. The autoregressive nature of X is, as expected, readily identified.

The numerical value of the transfer entropy is influenced by the choice of the quantiles. Naturally, changing the quantiles leads to an increase or decrease in the number of the observations in the tails and the center of the empirical distribution of the data. Thus, the symbolic encoding is different and the probabilities in Eqs. (3) and (4) change accordingly, which may lead to a different numerical value of the transfer entropy. Since the transfer entropy is not normalized to a certain interval, it is clear that the information flow in both directions cannot be compared across different quantiles but only for the same quantiles, i.e. determining the dominant direction of the information flow is not possible across different quantile choices. To illustrate this effect, we re-estimate Shannon transfer entropy for a selection of quantiles. Fig. 1 reports the results for increasing tail bins (and, thus, a shrinking central bin). As can be seen, the numerical value of the effective transfer entropy from X to Y increases as the central bin gets smaller. Statistical significance is, however, not affected. Moreover, in this particular simulation experiment, the confidence intervals are fairly narrow across the different quantiles, since the estimated standard errors are small. For the first 5% and 95% quantiles, the standard errors are given in the first row of Table 2.

A different approach to calculate transfer entropy is provided by Rényi transfer entropy, which allows to weight the probabilities associated with the individual bins. Due to the proposed symbolic encoding of the data, tail events are associated with a lower likelihood compared to events in the center of the distribution. Thus, Rényi transfer entropy puts more weight on the tails when calculating transfer entropy. This is particularly convenient when the distribution is assumed to be more informative in the tails. Note again that Rényi transfer entropy estimates can potentially turn out negative.

To illustrate the use of Rényi transfer entropy, we simulate data for which the dependence of Y on X changes with the size of the innovation.

$$x_t = 0.2x_{t-1} + \varepsilon_{x,t} \tag{9}$$
$$y_t = \begin{cases} x_{t-1} + \varepsilon_{y,t} & \text{if } |\varepsilon_{y,t}| > s \\ 0.2x_{t-1} + \varepsilon_{y,t} & \text{if } |\varepsilon_{y,t}| < s, \end{cases}$$

with $\varepsilon_{x,t}$ and $\varepsilon_{y,t}$ being standard normally distributed such that $\sigma_y = 1$, and $s = 2\sigma_y$. As before, X serves as a predictor for Y, but not vice versa.

```
R> set.seed(12345)
R> n <- 10000
R> x <- rep(0, n + 200)
R> y <- rep(0, n + 200)
R>
R> x[1] <- rnorm(1, 0, 1)
R> y[1] <- rnorm(1, 0, 1)
R>
R> for (i in 2:(n + 200)) {
R+  x[i] <- 0.2 * x[i - 1] + rnorm(1, 0, 1)
R+  y[i] <- ifelse(
R+    abs(x[i - 1]) > 1.65,
R+    x[i - 1]  + rnorm(1, 0, 1),
R+    0.2 * x[i - 1] + rnorm(1, 0, 1)
R+  )
R+ }
R>
R> x <- x[-(1:200)]
R> y <- y[-(1:200)]
```

We estimate Rényi transfer entropy with a weighting parameter $q = 0.3$, which gives a reasonable weight to the (infrequent) tail observations. Estimation of Rényi transfer entropy is invoked as follows:

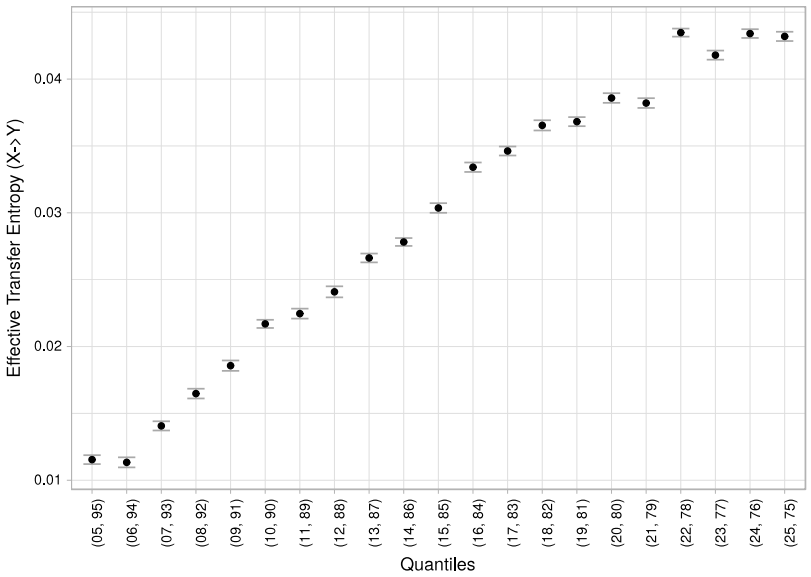

Fig. 1. Effective Shannon transfer entropy values for different quantiles.

```
R> renyi_te <- transfer_entropy(x, y,
R+              entropy = "Renyi", q = 0.3, seed = 12345)

Renyi's entropy on 8 cores with 100 shuffles.
  x and y have length 10000 (0 NAs removed)
  [calculate] X->Y transfer entropy
  [calculate] Y->X transfer entropy
  [bootstrap] 300 times
Done - Total time 14.41 seconds
```

The results, presented in Table 4, indicate that there is statistically significant information flow from X to Y. The effect in the other direction is not statistically significant.

6. Application to financial time series

The analysis of information flows in financial markets has a long history. Using transfer entropy widens the possibilities to detect information flows as nonlinear relationships can also be accounted for. To illustrate the application of transfer entropy, we use a dataset of 10 individual stocks comprised in the S&P 500 index as well as the index itself.[5] The data range from January 3, 2000, to December 29, 2017. This dataset is included as the `stocks` object in the package. As the calculation of transfer entropy requires stationary data, we calculate log-returns from the price series.

A core question in finance is how individual stocks are priced. The Capital Asset Pricing Model, for example, seeks to explain stock price movements using the market return. Hence, it captures the linear correlation between the market and the individual stock, which in turn might be used to predict returns of the stock when market changes are known. Using transfer entropy, we can quantify the information flow from the market to the individual stocks and thereby determine to which extent individual stocks are susceptible to information that is due to changes in the market environment as a whole.

The stock market index itself is a weighted average of individual stocks. Of course, small stocks have less weight and, thus, may only have limited impact on the index, while large corporations might dominate. Transfer entropy can be used to unveil the information that flows from the individual companies' returns to the market.

Fig. 2 reports effective transfer entropy estimates together with 95% confidence bounds for 10 stocks. Following [34], the return series may be modeled as Markov processes of order one, which is why we leave `lx` and `ly` at the default value. The results illustrate that, indeed, a bi-directional information flow is present. However, the information flow from the stocks to the market is higher for most stocks than the other direction. Also, the information flow towards the stock is not statistically significant in two cases (on a 5% significance level).

In finance, pricing relevant information is readily associated with tail events, which refer to relatively large positive or negative returns. If these are indeed more relevant, Rényi transfer entropy provides a tool to give more weight to their contribution to the overall information flow. To illustrate this feature we use the AXP stock and calculate Rényi transfer entropy between the S&P 500 index and the stock for a selection of the weighting parameters q. The result is graphically depicted in Fig. 3.

For low values of q, the information in the tails is given a larger weight, which leads in the current situation to a significant effective transfer entropy result. This indicates that, indeed, tail events are rather informative and help to predict the movements of the S&P 500 and the AXP stock, respectively. As the weight on the tails is reduced, the effective transfer entropy decreases and even becomes negative (between $q = 0.3$ and $q = 0.8$ in the direction from the stock to the index and between $q = 0.5$ and $q = 0.9$ in the other direction). Such a situation implies that knowledge of either the stock or the index development indicates a higher risk exposure of the respective other entity. Note that this does not mean that there is no information flow. Rényi transfer entropy merely suggests a higher risk of the predicted variable as opposed to a situation with positive Rényi transfer entropy where the risk about the future returns of the predicted variable is reduced by knowledge of the present returns of the other variable.

The dashed line in the graph represents the Shannon transfer entropy. The last value of q is 0.99 for which the Rényi transfer entropy is already fairly close to the value of the Shannon transfer

[5] ADBE Adobe Systems Incorporated; AES AES Corporation; ALK Alaska Air Group, Inc.; AMD Advanced Micro Devices, Inc.; AMZN Amazon.com, Inc.; APD Air Products and Chemicals, Inc.; ARE Alexandria Real Estate Equities, Inc.; AXP American Express Company; BXP Boston Properties, Inc.; IBM International Business Machines Corporation.

Table 4
Rényi transfer entropy result.

Panel A: Transfer entropy estimates					
Direction	TE	Eff. TE	Std. Err.	p-value	sig
X → Y	0.1121	0.0654	0.0304	0.0233	*
Y → X	0.0245	−0.0280	0.0253	0.8333	
Panel B: Bootstrapped TE quantiles					
Direction	0%	25%	50%	75%	100%
X → Y	−0.0145	0.0293	0.0501	0.0647	0.1224
Y → X	−0.0224	0.0264	0.0466	0.0651	0.1437

Note. The table presents the Rényi transfer entropy estimates based on data simulated according to Eq. (9). The estimation (Panel A) is based on 10,000 observations and a weighting parameter $q = 0.3$. Bootstrapped transfer entropy quantiles (Panel B) are based on 300 bootstrap replications. *** (**, *) indicates significance on a 0.1% (1%, 5%) significance level.

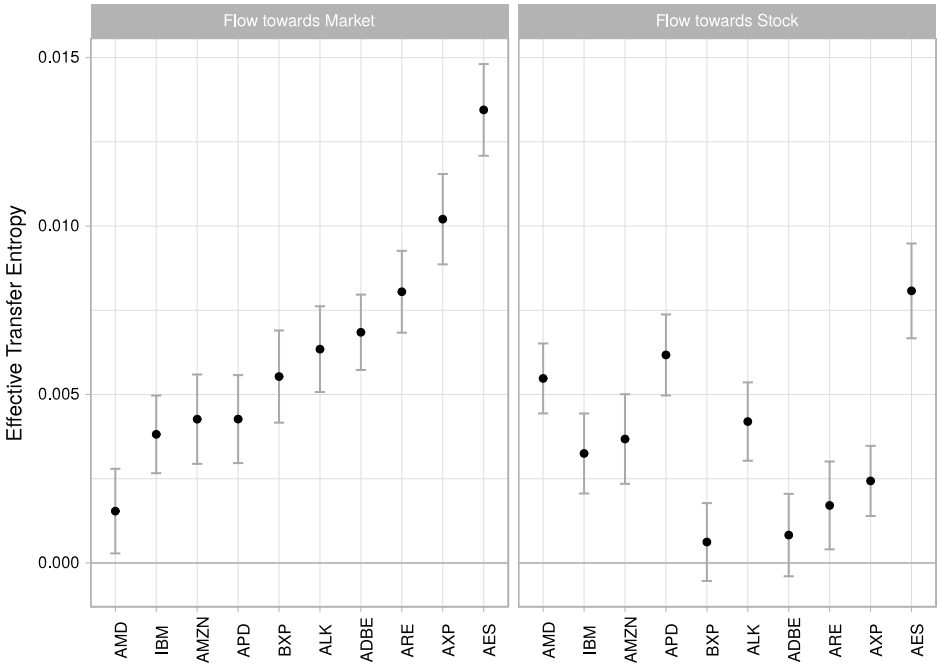

Fig. 2. Estimated Shannon transfer entropy values for selected Stocks.

entropy. This illustrates once more that Rényi transfer entropy converges to Shannon transfer entropy as q approaches 1.

7. Conclusions

We present and outline empirical as well as theoretical aspects concerning the *R* package *RTransferEntropy*, which provides a convenient tool to estimate Shannon and Rényi transfer entropy, both well-established measures derived from information theory. In particular, the package calculates effective transfer entropy using a simple finite sample bias correction and provides the means to conduct statistical inference, which is an important aspect in every application across research areas.

Since transfer entropy is a nonparametric measure and detects any statistical dependence between time series, it is applied in various disciplines such as economics, biometrics, or neuroscience. We demonstrate the functionality of the package by means of multiple examples based on simulated processes as well as an empirical application to financial time series. The latter illustrates how transfer entropy can be used to detect interdependencies between individual stocks and the market as a whole and how Rényi transfer entropy allows to give more weight to tail events in their contribution to the overall information flow.

Acknowledgments

We acknowledge support by the German Research Foundation (Deutsche Forschungsgemeinschaft DFG) and the Open Access Publishing Fund of the University of Tübingen.

Declaration of competing interest

We wish to confirm that there are no known conflicts of interest associated with this publication and there has been no significant financial support for this work that could have influenced its outcome.

Appendix. Proof of entropy convergence

Proposition:

For $q \longrightarrow 1$, Rényi entropy, given by $H_J^q = \frac{1}{1-q} \log \sum_j p^q(j)$ and where log denotes the base two logarithm, converges to Shannon entropy, which is given by $H_J = -\sum_j p(j) \log p(j)$.

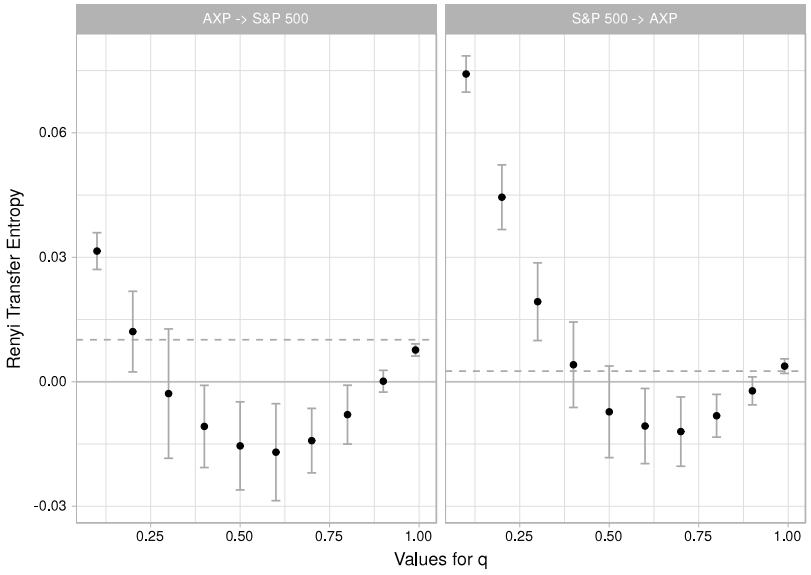

Fig. 3. Rényi transfer entropy for different values of q.

Proof:

Define two continuous and differentiable functions over the open interval $(0, 1)$, $f : (0, 1) \longrightarrow \mathbb{R},\ q \longrightarrow f(q) = \log \sum_j p^q(j)$ and $g : (0, 1) \longrightarrow \mathbb{R},\ q \longrightarrow g(q) = 1 - q$, then Rényi entropy is given by $H_j^q = \frac{f(q)}{g(q)}$. Since $\lim_{q\to 1} f(q) = 0$ and $\lim_{q\to 1} g(q) = 0$, the rule of l'Hospital gives

$$\begin{aligned}
\lim_{q\to 1} H_j^q &= \lim_{q\to 1} \frac{f(q)}{g(q)} \\
&= \lim_{q\to 1} \frac{f(q)'}{g(q)'} \\
&= \lim_{q\to 1} (-1) \frac{\sum_j p^q(j) \ln p(j)}{\ln 2 \sum_j p^q(j)} \\
&= (-1) \frac{\sum_j p(j) \ln p(j)}{\ln 2 \sum_j p(j)} \\
&= -\sum_j p(j) \frac{\ln p(j)}{\ln 2} \\
&= -\sum_j p(j) \log p(j) = H_J. \quad \blacksquare
\end{aligned}$$

References

[1] Schreiber T. Measuring information transfer. Phys Rev Lett 2000;85(2):461–4. http://dx.doi.org/10.1103/PhysRevLett.85.461.

[2] Yao W, Wang J. Multi-scale symbolic transfer entropy analysis of EEG. Physica A 2017;484:276–81. http://dx.doi.org/10.1016/j.physa.2017.04.181.

[3] Oh M, Kim S, Lim K, Kim SY. Time series analysis of the antarctic circumpolar wave via symbolic transfer entropy. Physica A 2018. http://dx.doi.org/10.1016/j.physa.2017.12.019.

[4] Dimpfl T, Peter FJ. The impact of the financial crisis on transatlantic information flows: an intraday analysis. J Int Financ Markets Institutions Money 2014;31:1–13. http://dx.doi.org/10.1016/j.intfin.2014.03.004.

[5] Lee J, Nemati S, Silva I, Edwards BA, Butler JP, Malhotra A. Transfer entropy estimation and directional coupling change detection in biomedical time series. Biomed Eng Online 2012;11(1):19. http://dx.doi.org/10.1186/1475-925X-11-19.

[6] Zheng L, Pan W, Li Y, Luo D, Wang Q, Liu G. Use of mutual information and transfer entropy to assess interaction between parasympathetic and sympathetic activities of nervous system from HRV. Entropy 2017;19:489. http://dx.doi.org/10.3390/e19090489.

[7] Dimitrov AG, Lazar AA, Victor JD. Information theory in neuroscience. J Comput Neurosci 2011;30(1):1–5. http://dx.doi.org/10.1007/s10827-011-0314-3.

[8] Amblard P-O, Michel OJ. On directed information theory and granger causality graphs. J Comput Neurosci 2011;30(1):7–16. http://dx.doi.org/10.1007/s10827-010-0231-x.

[9] Vicente R, Wibral M, Lindner M, Pipa G. Transfer entropy–A model-free measure of effective connectivity for the neurosciences. J Comput Neurosci 2011;30(1):45–67. http://dx.doi.org/10.1007/s10827-010-0262-3.

[10] Borge-Holthoefer J, Perra N, Gonçalves B, González-Bailón S, Arenas A, Moreno Y, Vespignani A. The dynamics of information-driven coordination phenomena: A transfer entropy analysis. Sci Adv 2016;2(4). e1501158. http://dx.doi.org/10.1126/sciadv.1501158.

[11] Bossomaier T, Barnett L, Harr M, Lizier JT. An introduction to transfer entropy: information flow in complex systems. first ed.. Springer International Publishing; 2016, http://dx.doi.org/10.1007/978-3-319-43222-9.

[12] Behrendt S, Dimpfl T, Peter F, Zimmermann D. *RTransferEntropy*: measuring information flow between time series with Shannon and Renyi transfer entropy, *R* package version 0.2.7. 2018, https://CRAN.R-project.org/package=RTransferEntropy.

[13] Hartley RV. Transmission of information. Bell Labs Tech J 1928;7(3):535–63. http://dx.doi.org/10.1002/j.1538-7305.1928.tb01236.x.

[14] Dimpfl T, Peter FJ. Using transfer entropy to measure information flows between financial markets. Stud Nonlinear Dyn Econom 2013;17(1):85–102. http://dx.doi.org/10.1515/snde-2012-0044.

[15] Shannon CE. A mathematical theory of communication. Bell Labs Tech J 1948;27:379–423. http://dx.doi.org/10.1002/j.1538-7305.1948.tb01338.x.

[16] Kullback S, Leibler RA. On information and sufficiency. Ann Math Stat 1951;1:79–86. http://dx.doi.org/10.1214/aoms/1177729694.

[17] Rényi A. Probability theory. Amsterdam: North-Holland; 1970, http://dx.doi.org/10.1002/zamm.19710510718.

[18] Jizba P, Kleinert H, Shefaat M. Renyi's information transfer between financial time series. Physica A 2012;391:2971–89. http://dx.doi.org/10.1016/j.physa.2011.12.064.

[19] Beck C, Schögl F. Thermodynamics of chaotic systems: an introduction. In: Cambridge nonlinear science series, (vol. 4). Cambridge University Press; 1993, http://dx.doi.org/10.1017/CBO9780511524585.

[20] Marschinski R, Kantz H. Analysing the information flow between financial time series: An improved estimator for transfer entropy. Eur Phys J B 2002;30(2):275–81. http://dx.doi.org/10.1140/epjb/e2002-00379-2.

[21] Eddelbuettel D, Francois R, Allaire J, Ushey K, Kou Q, Russell N, Bates D, Chambers J. *Rcpp*: seamless *R* and *C++* integration, *R* package version 0.12.18. 2018, https://CRAN.R-project.org/package=Rcpp.

[22] Bengtsson H. *Future*: unified parallel and distributed processing in R for everyone, *R* package version 1.8.1. 2018, https://CRAN.R-project.org/package=future.

[23] Hegger R, Kantz H. Improved false nearest neighbor method to detect determinism in time series data. Phys Rev E 1999;60(4):4970–3.

[24] Cao L. Practical method for determining the minimum embedding dimension of a scalar time series. Physica D 1997;110:43–50.
[25] Kantz H, Schreiber T. Nonlinear time series analysis. second ed.. New York: Cambridge University Press; 2004.
[26] Fabio Di Narzo A. *tseriesChaos*: ANalysis of nonlinear time series, *R* package version 0.1-13.1. 2019, https://CRAN.R-project.org/package=tseriesChaos.
[27] Garcia CA, Sawitzki G. *nonlinearTseries*: Nonlinear time series analysis, *R* package version 0.2.6. 2019, https://CRAN.R-project.org/package=nonlinearTseries.
[28] Montalto A, Faes L, Marinazzo D. MuTE: a MATLAB Toolbox to compare established and novel estimators of the multivariate transfer entropy. PLoS One 2014;9(10). e109462.
[29] Lizier JT. JIDT: An Information-theoretic toolkit for studying the dynamics of complex systems. Frontiers Robotics AI 2014;1:11.
[30] Wollstadt P, Lizier JT, Vicente R, Finn C, Martınez-Zarzuela M, Mediano P, Novelli L, Wibral M. IDTxl: The Information Dynamics Toolkit xl: a Python Package for the efficient analysis of multivariate information dynamics in networks. PloS One 2013;8(2). e55809.
[31] Lindner M, Vicente R, Priesemann V, Wibral M. TRENTOOL: A matlab open source toolbox to analyse information flow in time series data with transfer entropy. BMC Neurosci 2011;12(1):119.
[32] Granger CWJ. Investigating causal relations by econometric modles and cross-spectral methods. Econometrica 1969;37:424–38.
[33] Pfaff B, Stigler M. *vars*: VAR modelling, *R* package version 1.5-3. 2018, https://CRAN.R-project.org/package=vars.
[34] Turner CM, Startz R, Nelson CR. A Markov model of heteroskedasticity, risk, and learning in the stock market. J Financ Econ 1989;25(1):3–22.